UNIVERSIDADE FEDERAL DE MINAS GERAIS
INSTITUTO DE CIÊNCIAS EXATAS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
PROJETO DE SISTEMAS ROBÓTICOS — 2º SEMESTRE DE 2026

Relatório 5

ROS e navegação

Desvio de obstáculos e navegação até metas com o robô HeRo em simulação

Equipe

Ari Gonçalves da Silva Filho
Caio Rodrigues Costa
Cleiton Neves Santos
Túlio Henrique Rodrigues Costa

Professor: Paulo Rezeck

Repositório: https://github.com/TulioHRC/robotica-cain

Belo Horizonte, 17 de novembro de 2026

Universidade Federal de Minas Gerais
Instituto de Ciências Exatas
Departamento de Ciência da Computação

Projeto de Sistemas Robóticos
2º semestre de 2026
Prof. Paulo Rezeck

Relatório 5: ROS e navegação

Entrega	17/11/2026
Repositório	`https://github.com/TulioHRC/robotica-cain`
Vídeos	`https://youtube.com/playlist?list=PLI7BuXk2BAFs`
Ambiente	Kit `psr-hero-kit`, ROS 1 Noetic, Genesis (versão GPU, Linux com NVIDIA)

1. Introdução

Este relatório descreve a implementação e a avaliação de dois comportamentos para o robô HeRo [1, 2], escritos como nós de ROS 1 Noetic [3] e testados no simulador do kit da disciplina, que usa o motor de física Genesis [4]. O primeiro nó, `avoid`, mantém o robô andando pela arena sem colidir, usando apenas um lidar de oito feixes. O segundo, `goto`, conduz o robô até metas sorteadas pelo simulador, usando a pose da odometria. Como os tópicos do simulador são os mesmos do robô real, o código desenvolvido aqui é a base para os experimentos com o HeRo físico.

Desviar de obstáculos e ir até um ponto são dois comportamentos básicos de um robô móvel. O primeiro protege o robô sem exigir mapa nem localização, e o segundo é a base de qualquer tarefa com destino definido. Tarefas maiores, como as do projeto final, combinam os dois, por exemplo indo até uma meta e desviando do que estiver no caminho.

Nos ensaios, a configuração A do `avoid` percorreu em média 38,2 m em 5 min, sem nenhuma colisão. A configuração B, mais agressiva, percorreu 51,5 m, também sem colisões, mas com margem menor até os obstáculos. O `goto` atingiu em média 33,3 metas em 3 min com o conjunto de ganhos A, com 5,32 s por meta, contra 28,3 metas com o conjunto B.

O restante do texto está organizado da seguinte forma. A Seção 2 apresenta os nós, os tópicos e o registro dos dados. As Seções 3 e 4 tratam de cada exercício, com a lei de controle, o ensaio, os resultados e a comparação entre duas configurações. A Seção 5 discute as limitações, as falhas observadas e a transferência para o robô real, e a Seção 6 conclui o trabalho.

2. Arquitetura do sistema

2.1 Simulador, ponte e nós

O kit divide o sistema em duas partes que conversam por TCP (Figura 1). O simulador calcula a física, desenha a janela 3D e faz o papel de juiz do cenário. O nó `hero_bridge` converte o estado do simulador em mensagens ROS e repassa ao simulador os comandos de velocidade. Os nós da equipe, `avoid_0` e `goto_0`, são iniciados pelos arquivos `avoid.launch` e `goto.launch`, que definem o parâmetro privado `~robot` igual a 0.

Os tópicos são publicados a 25 Hz e os dois nós executam o laço de controle a 20 Hz. O simulador satura os comandos em $|v| \leq 0{,}2\,\mathrm{m/s}$ e $|\omega| \leq 4\,\mathrm{rad/s}$ e para o robô se nenhum comando chegar em 0,5 s, como faz o firmware do robô real. A Tabela 1 resume os tópicos usados.

2.2 Registro dos ensaios

Cada nó grava um arquivo CSV na subpasta `logs/` do próprio script (`exercicios/avoid/logs/` ou `exercicios/goto/logs/`), que é compartilhada entre o container e o computador da equipe. O nome do arquivo traz a configuração e a hora de início, por exemplo `avoid_A_20261110-143000.csv`. Uma linha é

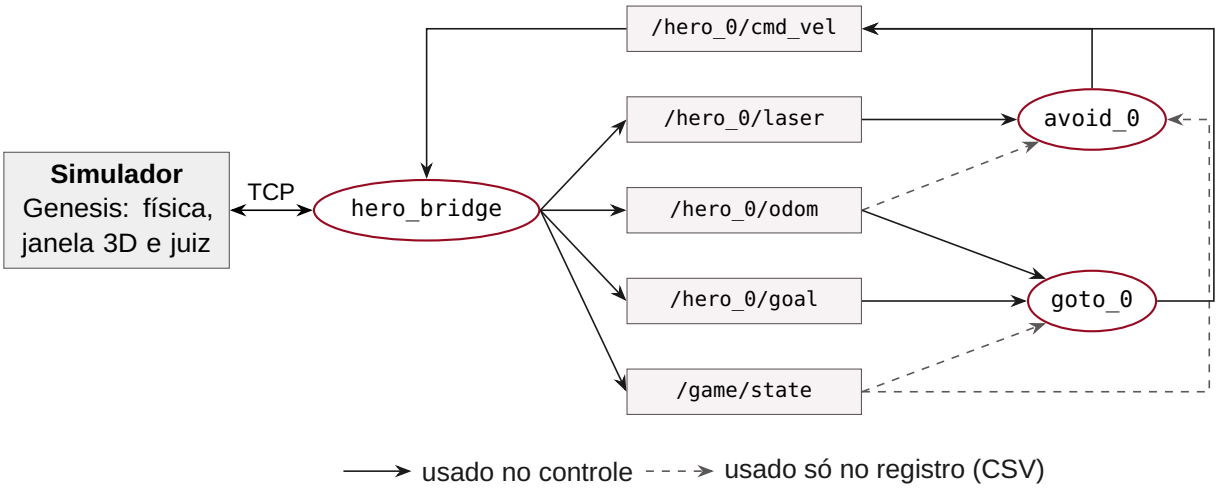


Figura 1. Nós (elipses) e tópicos (retângulos) do sistema. O tópico `/hero_0/goal` só existe no cenário `goto`. No `avoid`, a odometria e o `/game/state` são assinados apenas para o registro do ensaio.

Tabela 1. Tópicos usados pelos nós da equipe.

Tópico	Tipo	Assinado por	Uso
`/hero_0/laser`	`sensor_msgs/LaserScan`	`avoid_0`	8 distâncias em `msg.ranges`, entrada do controle
`/hero_0/odom`	`nav_msgs/Odometry`	ambos	pose (x, y, θ), controle no `goto` e só registro no `avoid`
`/hero_0/goal`	`geometry_msgs/PointStamped`	`goto_0`	meta atual (x_g, y_g)
`/game/state`	`std_msgs/String`	ambos	JSON com o tempo simulado `t` e, no `goto`, as metas atingidas `reached`. Usado só no registro
`/hero_0/cmd_vel`	`geometry_msgs/Twist`	`hero_bridge`	comando publicado pelos nós: `linear.x` e `angular.z`

escrita a cada iteração do laço de controle, isto é, a 20 Hz. O tempo t é o tempo simulado, lido do campo `t` de `/game/state`, e começa a contar na primeira iteração em que o nó já recebeu todos os tópicos. Usar o relógio do simulador, e não o do computador, mantém as durações e as velocidades corretas mesmo se a simulação rodar mais devagar que o tempo real. O nó encerra o ensaio sozinho depois de 300 s no `avoid` e 180 s no `goto`, publicando um comando nulo antes de sair. A Tabela 2 lista as colunas. Os gráficos e as métricas são gerados fora do container pelos scripts descritos no Apêndice A, que descartam ensaios interrompidos antes do fim.

Tabela 2. Colunas dos arquivos de registro.

Coluna	Descrição	Unidade	Exercício
`t`	tempo simulado desde o início do ensaio	s	ambos
`x`, `y`, `theta`	pose do robô na odometria	m, rad	ambos
`d_min`	menor das 8 distâncias do lidar	m	`avoid`
`d0` a `d7`	distâncias de cada feixe	m	`avoid`
`xg`, `yg`	meta atual	m	`goto`
`rho`, `alpha`	distância e erro de orientação até a meta	m, rad	`goto`
`metas`	metas atingidas, lido de `/game/state`	–	`goto`
`v`, `w`	comando publicado	m/s, rad/s	ambos
`escape`	1 se o modo de escape está ativo	–	`avoid`

3. Desvio de obstáculos (`avoid`)

3.1 Arena e sensor

A arena é um quadrado de 2,0 × 2,0 m centrado na origem, cercado por paredes, com um cilindro de raio 0,15 m no centro. O robô começa em $x = -0{,}7\,\mathrm{m}$ e $y = 0$, voltado para $+x$. O lidar tem 8 feixes, e o feixe k aponta para o ângulo $-\pi + k\pi/4$ em relação à frente do robô, no sentido anti-horário (Figura 2). O alcance vai de 0,037 m a 0,5 m, um feixe que não encontra nada mede 0,5 m, e as distâncias são medidas a partir do centro do robô, cujo raio é 0,037 m. Assim, o critério de colisão do enunciado, $d_{\min} < 0{,}045\,\mathrm{m}$, corresponde a menos de 8 mm entre o corpo do robô e o obstáculo. Pela regra do exercício, o comando é calculado apenas com o lidar.

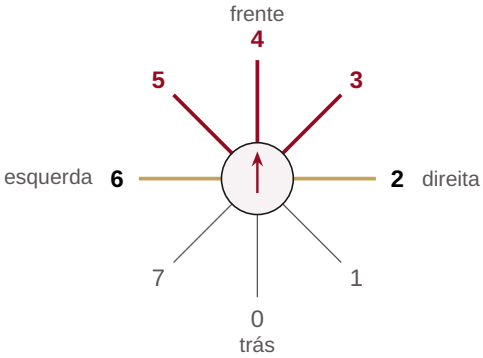


Figura 2. Numeração dos feixes do lidar. Em vermelho, os feixes de d_F (frente). Os feixes 5 e 6 formam d_E (esquerda) e os feixes 2 e 3 formam d_D (direita).

3.2 Lei de controle

A lei segue as dicas do README do exercício. A velocidade linear cai quando há obstáculo à frente, o robô gira para o lado mais livre e um modo de escape com histerese evita que ele oscile em cantos. Sejam d_k as distâncias medidas pelos feixes. Cada leitura é primeiro saturada no alcance de influência d_{inf}, de

modo que obstáculos mais distantes que isso são tratados como espaço livre. Com as leituras saturadas $\bar{d}_k$, a lei usa três grandezas:

$$\bar{d}_k = \min(d_k, d_{\text{inf}}), \qquad d_F = \min(\bar{d}_3, \bar{d}_4, \bar{d}_5), \qquad d_E = \tfrac{1}{2}(\bar{d}_5 + \bar{d}_6), \qquad d_D = \tfrac{1}{2}(\bar{d}_2 + \bar{d}_3). \tag{1}$$

A velocidade linear cai linearmente de $v_{\max}$ a zero quando d_F passa de d_{lento} para d_{pare}, e a velocidade angular gira o robô para o lado com mais espaço livre:

$$v = v_{\max}\ \text{sat}_{[0,1]}\left(\frac{d_F - d_{\text{pare}}}{d_{\text{lento}} - d_{\text{pare}}}\right), \qquad \omega = \text{sat}_{[-\omega_{\max},\,\omega_{\max}]}\left(k_\omega\,(d_E - d_D) + b\right). \tag{2}$$

O termo b é um viés de vagar. A cada $T_v = 2\,\text{s}$ (40 iterações do laço), ele é sorteado de uma distribuição uniforme em $[-\omega_v, \omega_v]$, com semente fixa para que o ensaio possa ser repetido. Longe de obstáculos, $d_E = d_D = d_{\text{inf}}$ e só o viés atua, o que faz o robô descrever curvas suaves de raio variável. Perto deles, o termo $k_\omega(d_E - d_D)$ domina. O viés não usa nenhum sensor, então a regra de calcular o comando apenas com o lidar é mantida.

Quando $d_F < d_{\text{giro}}$, o robô entra no modo de escape e gira com $\omega = s\,\omega_e$, em que o sentido $s \in \{-1, +1\}$ é escolhido uma única vez, na entrada do modo, pelo sinal de $d_E - d_D$. O modo termina quando $d_F > d_{\text{livre}}$, e durante ele o viés não é aplicado. O código abaixo é o `compute_command` implementado, e a Tabela 3 traz os valores dos parâmetros nas duas configurações ensaiadas.

```
def compute_command(self, ranges):
    d = [min(r, self.D_INF) for r in ranges]
    d_f = min(d[3], d[4], d[5])
    d_e = (d[5] + d[6]) / 2.0
    d_d = (d[2] + d[3]) / 2.0
    f = (d_f - self.D_PARE) / (self.D_LENTO - self.D_PARE)
    v = self.V_LIN * clamp(f, 0.0, 1.0)
    if not self.escape and d_f < self.D_GIRO:
        self.escape = True
        self.s = 1 if d_e >= d_d else -1
    if self.escape:
        w = self.s * self.W_ESC
        self.escape = d_f <= self.D_LIVRE
    else:
        if self.k % self.T_VAG == 0:
            self.vies = self.rng.uniform(-self.W_VAG, self.W_VAG)
        self.k += 1
        w = self.K_W * (d_e - d_d) + self.vies
    return v, clamp(w, -self.W_LIM, self.W_LIM)
```

Os valores da configuração A deixam folga em relação ao critério de colisão. A frenagem começa a 0,30 m do centro do robô e o escape a 0,13 m, isto é, a cerca de 9 cm do seu corpo. Durante o projeto, uma simulação cinemática simples, com o modelo de lidar do próprio kit (`sim/psr_sim/geometry.py`) e um modelo de uniciclo, mostrou dois problemas das versões anteriores da lei. Sem a saturação em d_{inf}, isto é, com o alcance completo de 0,5 m, o robô se centralizava no corredor de 0,85 m entre o cilindro e as paredes e entrava em uma órbita fixa em torno do cilindro. Com $d_{\text{inf}} = 0{,}30\,\text{m}$, a órbita deu lugar a um ciclo de reflexões nas paredes, ainda periódico, que cobria cerca de 30 % da arena. O viés de vagar quebrou essa periodicidade e elevou a cobertura para cerca de 60 % na mesma simulação, sem colisões.

A configuração B é mais agressiva. A velocidade máxima sobe para 0,20 m/s, que é o limite do robô, e todas as distâncias de reação diminuem, de modo que o robô se aproxima mais dos obstáculos antes de frear e de girar. Na simulação cinemática, B percorreu cerca de 35 % a mais e cobriu cerca de 80 % da arena, com a menor distância caindo de cerca de 0,11 m para 0,07 m. Um ensaio de confirmação no

Tabela 3. Parâmetros do nó `avoid` nas duas configurações. Entre parênteses, o nome do atributo da classe `Avoid`.

Parâmetro	Símbolo	Config. A	Config. B	Unidade
Velocidade linear máxima (`V_LIN`)	$v_{\max}$	0,15	0,20	m/s
Distância para começar a frear (`D_LENTO`)	d_{lento}	0,30	0,20	m
Distância de parada (`D_PARE`)	d_{pare}	0,08	0,06	m
Ganho de giro (`K_W`)	k_ω	6,0	6,0	rad/(s m)
Velocidade angular máxima (`W_LIM`)	$\omega_{\max}$	3,0	3,0	rad/s
Velocidade angular de escape (`W_ESC`)	ω_e	2,5	2,5	rad/s
Entrada no modo de escape (`D_GIRO`)	d_{giro}	0,13	0,09	m
Saída do modo de escape (`D_LIVRE`)	d_{livre}	0,20	0,14	m
Alcance de influência (`D_INF`)	d_{inf}	0,30	0,20	m
Amplitude do viés de vagar (`W_VAG`)	ω_v	1,0	1,0	rad/s
Período do viés de vagar (`T_VAG`)	T_v	2,0	2,0	s

simulador, com a configuração A e $\omega_v = 0$, reproduziu o fenômeno (Figura 3). Depois de cerca de 30 s, o robô entrou em um circuito fechado, quase quadrado, e o manteve até o fim do ensaio, entre 0,62 m e 0,88 m do centro da arena, dando 7,5 voltas em torno do cilindro. A cobertura foi de 31,7 %, contra 56,8 % da configuração A com o viés (Tabela 4), e a distância percorrida foi praticamente a mesma (39,7 m). O viés, portanto, não muda o quanto o robô anda, apenas impede que a trajetória se feche em um ciclo.

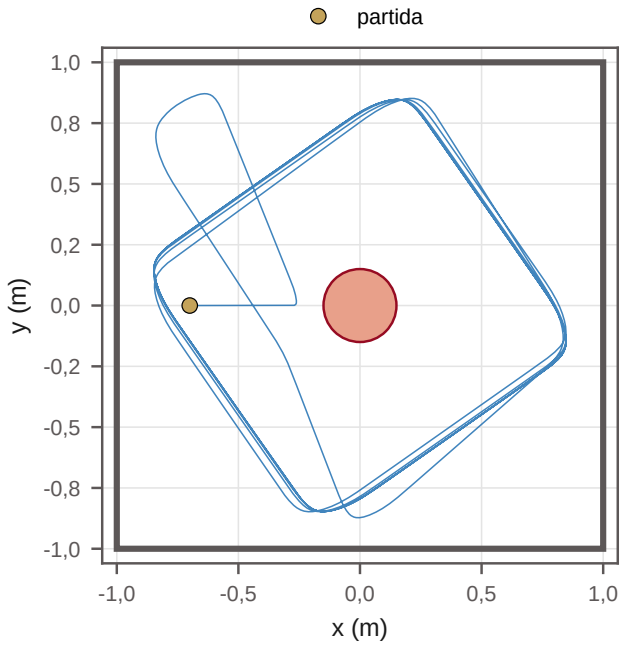


Figura 3. Trajetória de um ensaio de 300 s da configuração A sem o viés de vagar ($\omega_v = 0$). O robô fica preso em um circuito fechado em torno do cilindro.

3.3 Ensaio e métricas

Cada configuração foi ensaiada três vezes por 300 s, na versão GPU do simulador, em um computador com Linux e placa de vídeo NVIDIA. Os nomes dos arquivos de registro usam o horário UTC do container e por isso trazem a data de 4 de novembro. A simulação rodou em tempo real, com cerca de 6000 iterações do laço de controle por ensaio. O protocolo previa reiniciar o simulador antes de cada ensaio (`./run.sh down`), para que o robô partisse da pose de largada. Isso foi feito em três dos seis ensaios. Nos outros três (o terceiro de A e o segundo e o terceiro de B), o robô partiu da pose em que o ensaio anterior terminou.

Como a lei é reativa e o viés é sorteado de novo a cada execução do nó, a pose inicial não afeta as métricas de um ensaio de 5 min, e esses ensaios ficaram dentro da mesma faixa dos demais.

A distância percorrida é a soma dos deslocamentos entre amostras consecutivas da odometria, e uma colisão é contada a cada vez que $d_{\min}$ cruza o limiar de 0,045 m de cima para baixo, de modo que um contato prolongado conta como um único evento:

$$L = \sum_{i=1}^{n-1} \sqrt{(x_{i+1}-x_i)^2 + (y_{i+1}-y_i)^2}, \qquad N_{\text{col}} = \sum_{i=1}^{n-1} \mathbf{1}\left[d_{\min,i} \geq 0{,}045 \ \wedge \ d_{\min,i+1} < 0{,}045\right]. \tag{3}$$

Também são reportados a fração do tempo com $d_{\min} < 0{,}045\,\text{m}$, o menor valor de $d_{\min}$, a velocidade média L/T e a cobertura da arena. A cobertura é a fração das células de 10 × 10 cm que o centro do robô pode ocupar (fora do cilindro e afastadas das paredes por pelo menos o raio do robô) visitadas pelo menos uma vez. Quando há mais de um ensaio por configuração, a tabela de métricas mostra a média e o desvio padrão, e as figuras mostram o ensaio mais recente.

3.4 Resultados

A Figura 4 mostra a trajetória do ensaio da configuração A e a Figura 5 mostra $d_{\min}$ ao longo do tempo. As métricas das duas configurações estão na Tabela 4.

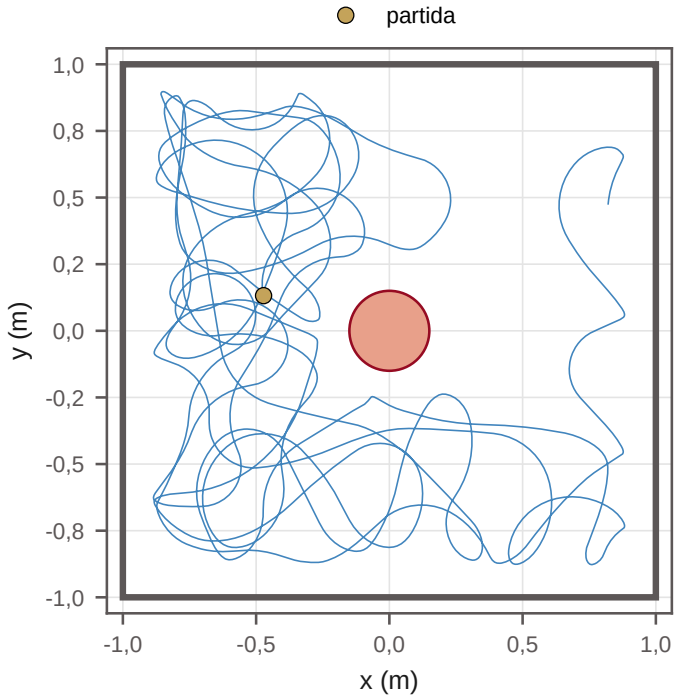


Figura 4. Trajetória do robô no ensaio mais recente da configuração A, que começou na pose em que o ensaio anterior terminou.

Tabela 4. Métricas dos ensaios de 300 s do nó `avoid`.

Métrica	Config. A	Config. B
Distância percorrida L (m)	x,xx	x,xx
Velocidade média L/T (m/s)	x,xx	x,xx
Número de colisões N_{col}	x,xx	x,xx
Tempo com $d_{\min} < 0{,}045\,\text{m}$ (%)	x,xx	x,xx
Menor $d_{\min}$ (m)	x,xx	x,xx
$d_{\min}$ médio (m)	x,xx	x,xx
Cobertura da arena (%)	x,xx	x,xx

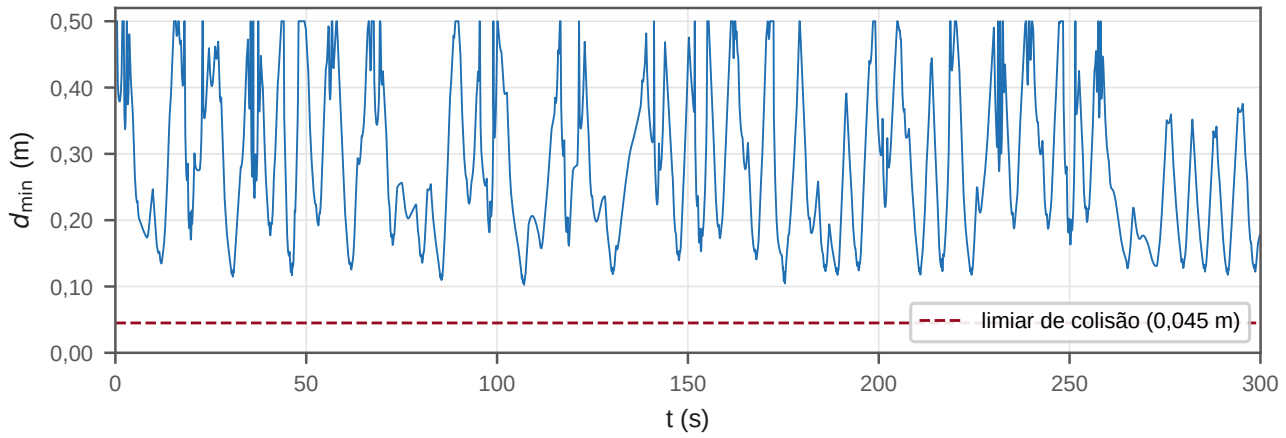
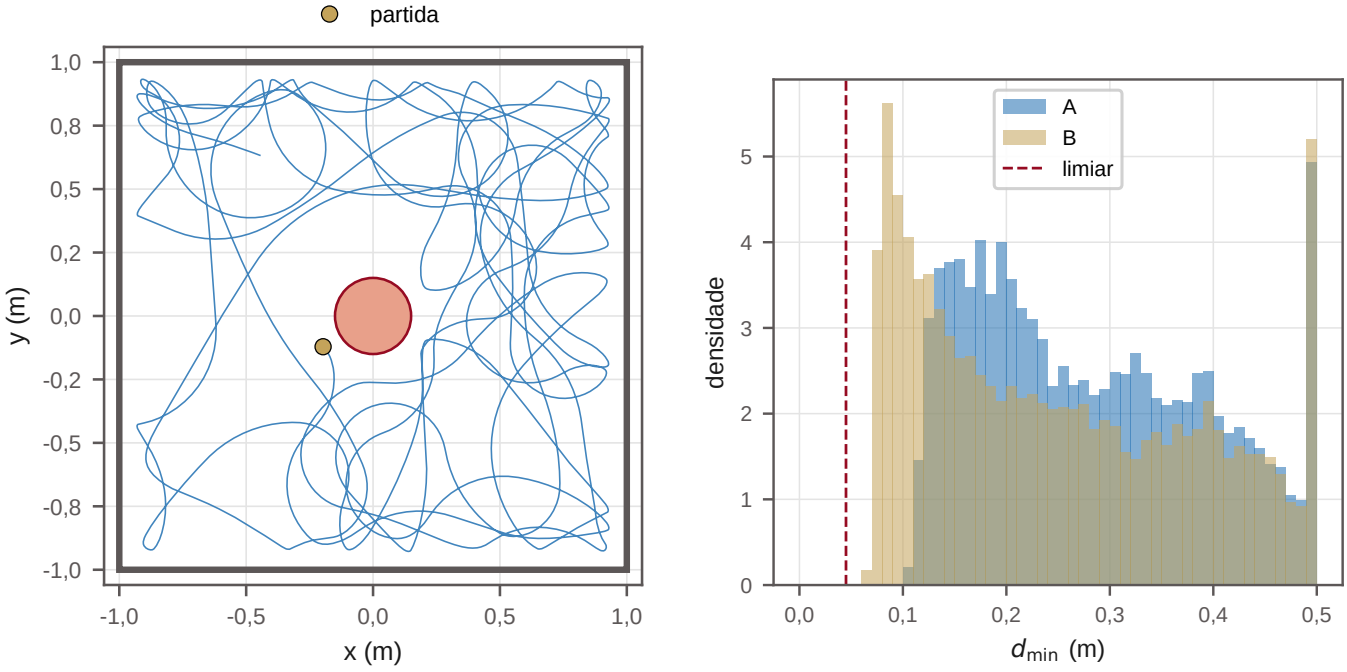


Figura 5. Menor distância lida pelo lidar ao longo do ensaio da configuração A.

Nos três ensaios, a configuração A não registrou nenhuma colisão, e $d_{\min}$ nunca ficou abaixo de 0,10 m. A menor leitura, 0,103 m, deixa cerca de 6,5 cm entre o corpo do robô e o obstáculo, bem acima do limiar de 0,045 m. A trajetória da Figura 4 percorre a arena em curvas suaves de raio variado, sem padrão periódico, e cobre em média 56,8 % das células livres. A cobertura não é uniforme. Ela chega a 74 % no meio da arena, mas cai para 34 % na faixa de 20 cm junto às paredes e para 15 % nos cantos, regiões que o robô evita por reagir a obstáculos a até $d_{\text{inf}} = 0{,}30\,\text{m}$. Na Figura 5, os vales de $d_{\min}$ correspondem às aproximações de paredes, cantos e cilindro, e os picos em 0,5 m aos trechos em que nenhum feixe vê obstáculo. O modo de escape foi acionado de 13 a 17 vezes por ensaio, em episódios de cerca de 0,5 s, a maioria diante de paredes (25 dos 44 episódios), seguida pelo cilindro (13) e pelos cantos (6). A velocidade média, 0,127 m/s, é 85 % de $v_{\max}$, e a diferença corresponde às frenagens perto dos obstáculos.

3.5 Comparação entre configurações



(a) Trajetória da configuração B.

(b) Distribuição de $d_{\min}$ em A e B.

Figura 6. Comparação entre as configurações A e B do nó `avoid`.

A configuração B percorreu 35 % a mais que a A (51,5 m contra 38,2 m) e cobriu 76,2 % da arena, contra 56,8 %. As duas diferenças vêm da velocidade máxima maior e das distâncias de reação menores, que fazem o robô frear mais tarde e circular mais perto de paredes e cantos. É justamente aí que está o ganho de cobertura: na faixa junto às paredes ela subiu de 34 % para 73 % e nos cantos de 15 % para 63 %, enquanto no meio da arena passou só de 74 % para 80 %.

O preço é a margem de segurança. A menor distância caiu de 0,106 m para 0,067 m em média, o que deixa só cerca de 2 cm até o limiar de colisão, e $d_{\min}$ ficou abaixo de 0,10 m em cerca de 14 % do tempo, contra nenhum instante em A. O histograma da Figura 6 mostra esse deslocamento: a distribuição de B começa logo acima de 0,06 m, enquanto a de A só começa em 0,10 m. B também acionou o escape com quase o dobro da frequência (83 episódios contra 44), sobretudo em paredes (51) e cantos (21).

Nenhuma das duas configurações colidiu, então ambas cumprem a tarefa pelo critério do enunciado. Para explorar a arena em um tempo fixo, B é melhor. Para operar com folga, e em especial para levar o código ao robô real, em que o ruído dos sensores e os atrasos consomem parte da margem, A é a escolha mais segura.

4. Navegação até metas (`goto`)

4.1 Arena e metas

A arena tem 2,0 × 2,0 m, sem obstáculos internos, e o robô começa em $x = y = -0{,}6\,\mathrm{m}$. A meta atual é publicada em `/hero_0/goal` e conta como atingida quando o centro do robô fica a menos de 6 cm dela. Nesse instante, o simulador sorteia uma nova meta dentro do quadrado de $\pm 0{,}75\,\mathrm{m}$, a mais de 0,6 m da posição do robô. O sorteio não tem semente fixa, então dois ensaios enfrentam sequências de metas diferentes. A pose usada no controle vem de `/hero_0/odom`.

4.2 Lei de controle

Com a pose (x, y, θ) e a meta (x_g, y_g), o nó calcula a distância ρ e o erro de orientação α até a meta [5], com α normalizado para $[-\pi, \pi)$ pela função `wrap_angle`:

$$\rho = \sqrt{(x_g - x)^2 + (y_g - y)^2}, \qquad \alpha = \mathrm{wrap}\big(\,\mathrm{atan2}(y_g - y,\, x_g - x) - \theta\big). \tag{4}$$

O controle é proporcional em α, e a velocidade linear é reduzida quando o erro de orientação é grande, chegando a zero quando a meta está atrás do robô:

$$\omega = \mathrm{sat}_{[-\omega_{\max},\,\omega_{\max}]}(k_\alpha\,\alpha), \qquad v = \mathrm{sat}_{[0,\,v_{\max}]}(k_\rho\,\rho)\;\max(0, \cos\alpha). \tag{5}$$

O ganho k_α gira o robô na direção da meta, e a saturação em $\omega_{\max}$ limita o giro para erros grandes. Com $k_\alpha = 3$, a saturação em 4 rad/s ocorre para $|\alpha| > 1{,}33\,\mathrm{rad}$. O fator $\max(0, \cos\alpha)$ zera a velocidade linear quando a meta está atrás do robô, para que ele gire no lugar em vez de se afastar dela, e reduz a velocidade de forma suave à medida que o erro de orientação cresce. O termo $k_\rho\,\rho$ só limita a velocidade quando $\rho < v_{\max}/k_\rho \approx 13\,\mathrm{cm}$, o que suaviza a chegada sem tornar a aproximação lenta, já que a meta conta a 6 cm. O conjunto B difere do A apenas no ganho de orientação, $k_\alpha = 1$, para isolar o efeito desse ganho. Na simulação cinemática usada no projeto, valores de k_α acima de 3 praticamente não mudaram o desempenho, porque ω já satura, enquanto $k_\alpha = 1$ reduziu o número de metas em cerca de 12 %.

Tabela 5. Ganhos e limites do nó `goto` nos dois conjuntos ensaiados. Os limites do robô são 0,2 m/s e 4 rad/s.

Parâmetro	Símbolo	Conjunto A	Conjunto B	Unidade
Ganho de orientação (`K_ALPHA`)	k_α	3,0	1,0	1/s
Ganho de distância (`K_RHO`)	k_ρ	1,5	1,5	1/s
Velocidade linear máxima (`V_LIN`)	$v_{\max}$	0,20	0,20	m/s
Velocidade angular máxima (`W_LIM`)	$\omega_{\max}$	4,0	4,0	rad/s

4.3 Ensaio e métricas

Cada conjunto de ganhos foi ensaiado por 180 s com o simulador reiniciado antes do ensaio, o que também zera o placar. Os instantes de chegada t_j são detectados pela troca da meta registrada nas colunas `xg` e

`yg`, e o script de análise confere essa contagem com o placar registrado na coluna `metas`. O tempo para atingir a meta j é $\Delta t_j = t_j - t_{j-1}$, com $t_0 = 0$, e o tempo médio é $\overline{\Delta t} = \frac{1}{M}\sum_{j=1}^{M} \Delta t_j$, em que M é o número de metas atingidas. Como as metas são sorteadas, também é reportado o tempo normalizado $\Delta t_j / \rho_{0,j}$, em que $\rho_{0,j}$ é a distância até a meta j no instante em que ela aparece, o que torna a comparação entre os conjuntos menos dependente do sorteio. Cada conjunto foi ensaiado três vezes, nas mesmas condições do `avoid`. Em quatro dos seis ensaios (o terceiro de A e os três de B), o simulador não foi reiniciado, e o robô partiu da pose e da meta pendente do ensaio anterior, com o placar já em 32, 66, 96 e 125 metas. As métricas não são afetadas, porque o script conta as metas pela troca da meta e pela diferença do placar desde o início do ensaio, e as duas contagens coincidiram nos seis ensaios.

4.4 Resultados

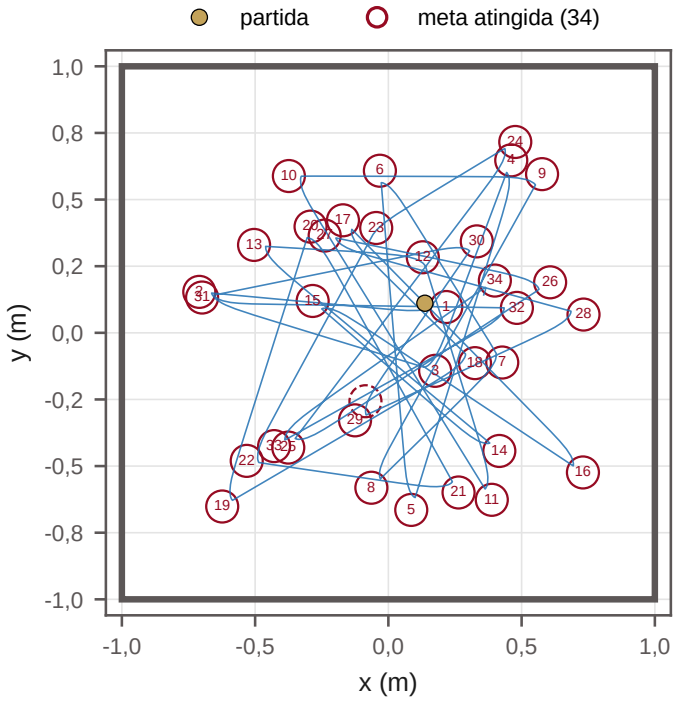


Figura 7. Trajetória do robô e metas atingidas no ensaio mais recente do conjunto A. O círculo tracejado é a meta pendente no fim do ensaio.

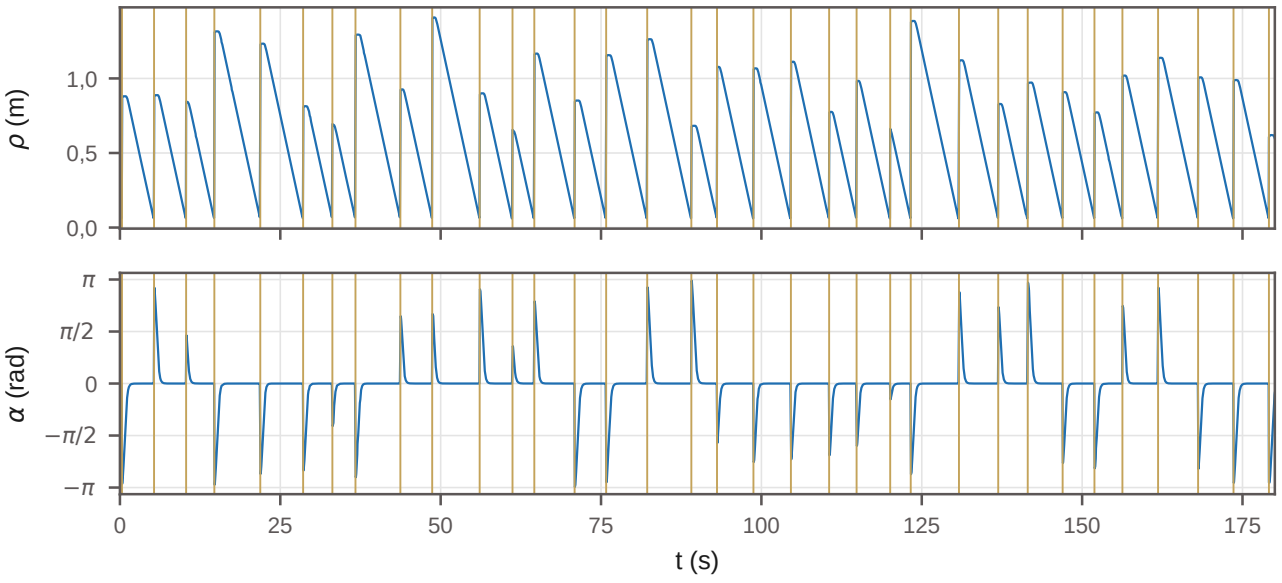


Figura 8. Distância ρ e erro de orientação α ao longo do ensaio do conjunto A.

O conjunto A atingiu 34, 32 e 34 metas nos três ensaios, com tempo médio de 5,32 s por meta e velocidade média de 0,177 m/s, ou 88 % de $v_{\max}$. A trajetória da Figura 7 é formada por segmentos quase retos entre

Tabela 6. Métricas dos ensaios de 180 s do nó `goto`.

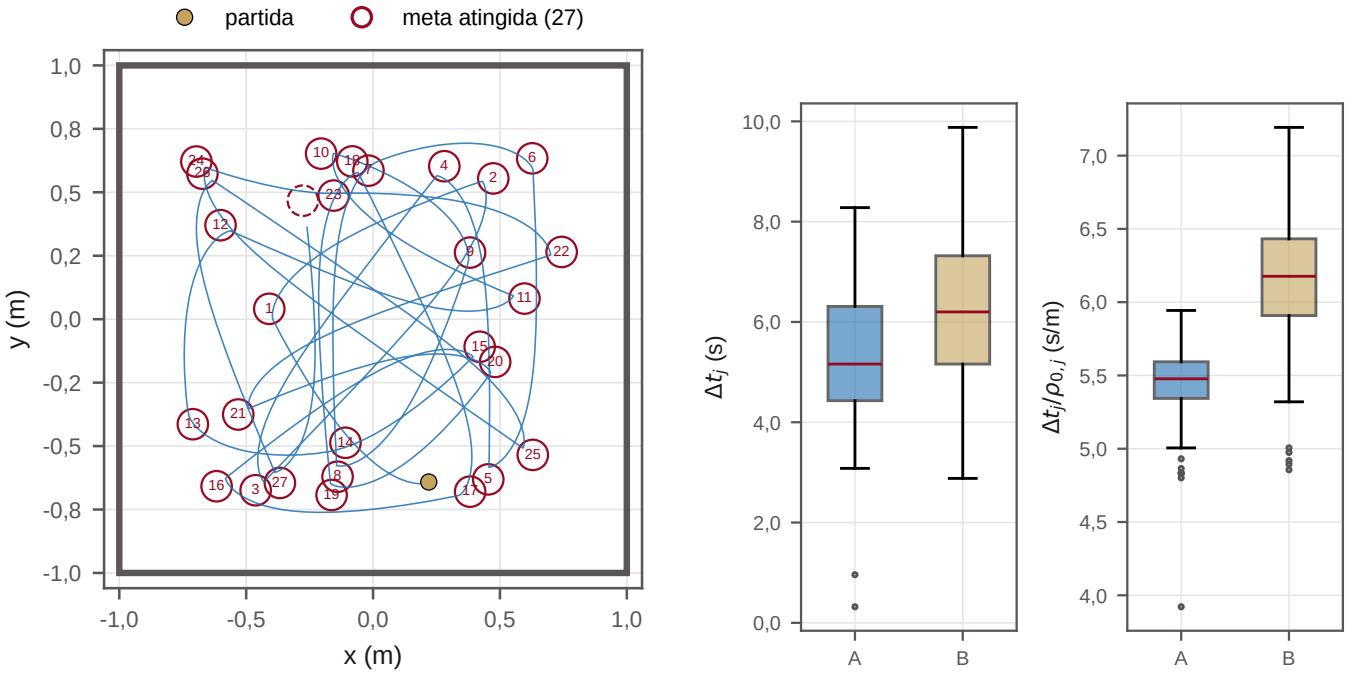
Métrica	Conjunto A	Conjunto B
Metas atingidas M	x,xx	x,xx
Tempo médio por meta $\overline{\Delta t}$ (s)	x,xx	x,xx
Desvio padrão de Δt_j (s)	x,xx	x,xx
Tempo normalizado médio $\Delta t_j / \rho_{0,j}$ (s/m)	x,xx	x,xx
Distância percorrida (m)	x,xx	x,xx

metas consecutivas, e o caminho percorrido até cada meta foi em média só 3 % mais longo que a distância em linha reta.

A Figura 8 mostra o padrão de cada meta. Quando uma meta nova aparece, α salta, muitas vezes para perto de $\pm\pi$, porque em 45 % dos casos a meta nova está atrás do robô ($|\alpha_0| > 2{,}5\,\mathrm{rad}$). O robô então gira quase no lugar, com ω saturado em 4 rad/s, e se alinha ($|\alpha| < 0{,}1\,\mathrm{rad}$) em média em 0,9 s. Depois disso, α fica próximo de zero e ρ decresce linearmente, com inclinação igual a $v_{\max}$, até a meta. A queda de ρ é praticamente monótona, e o aumento acumulado de ρ em uma meta não passou de 1,3 cm. Nos últimos 20 cm, α trocou de sinal no máximo duas vezes, em pequenas correções, sem oscilação sustentada nem órbita em torno da meta.

O tempo por meta é quase todo explicado pela distância. Um ajuste linear sobre as 100 metas dos três ensaios dá $\Delta t \approx 4{,}99\,\rho_0 + 0{,}29\,|\alpha_0| - 0{,}20$, com Δt em segundos, ρ_0 em metros e α_0 em radianos, e a correlação entre Δt e ρ_0 é de 0,99. O coeficiente de ρ_0 coincide com $1/v_{\max} = 5\,\mathrm{s/m}$, e o de $|\alpha_0|$ indica que cada radiano de erro inicial custa cerca de 0,3 s. Os dois tempos abaixo de 1 s na Figura 9 são metas que apareceram a menos de 0,2 m do robô.

4.5 Comparação entre conjuntos de ganhos



(a) Trajetória do conjunto B.

(b) Tempos por meta em A e B.

Figura 9. Comparação entre os conjuntos de ganhos A e B do nó `goto`.

Com $k_\alpha = 1$, o conjunto B atingiu 30, 28 e 27 metas, 15 % a menos que A, e o tempo médio por meta subiu 17 %, para 6,21 s. O tempo normalizado, que desconta a diferença de distância entre as metas sorteadas, subiu de 5,44 s/m para 6,12 s/m, e essa diferença é muito maior que a variação entre os ensaios de um

mesmo conjunto (Tabela 6).

A causa é o alinhamento. Com $k_\alpha = 1$, ω só saturaria para $|\alpha| > 4\,\mathrm{rad}$, o que nunca acontece, e o erro de orientação decai exponencialmente com constante de tempo $1/k_\alpha = 1\,\mathrm{s}$. O alinhamento até $|\alpha| < 0{,}1\,\mathrm{rad}$ levou em média 3,4 s, contra 0,9 s em A, o que concorda com a previsão $\ln(2{,}5/0{,}1) \approx 3{,}2\,\mathrm{s}$ para uma meta atrás do robô. Durante esse tempo o robô já anda, porque $\cos\alpha > 0$ assim que $|\alpha| < \pi/2$, e descreve os arcos largos visíveis na Figura 9. A eficiência do caminho caiu de 97 % para 93 %, chegando a 87 % no pior caso, e o custo de cada radiano de erro inicial quase dobrou, de 0,29 s para 0,53 s. A distância percorrida também foi menor em B (29,7 m contra 31,8 m), porque o robô passa mais tempo alinhando com a velocidade linear reduzida por $\cos\alpha$. Por outro lado, com o ganho menor, α não trocou de sinal nenhuma vez perto das metas.

O conjunto A é, portanto, melhor em todos os critérios de desempenho medidos. Como mostrou a simulação cinemática, ganhos maiores que 3 pouco acrescentariam, porque o giro já satura, e o tempo por meta está limitado pela velocidade linear máxima do robô.

5. Discussão

5.1 Limitações do desvio de obstáculos

A estratégia é puramente reativa: o comando depende só da leitura atual dos 8 feixes, sem memória e sem objetivo. A primeira consequência é que a dinâmica tende a ciclos fechados, como mostrou o ensaio sem viés. O viés de vagar resolve o sintoma, mas não garante cobertura completa, que depende do acaso e ficou em cerca de 57 % na configuração A, com as faixas junto às paredes pouco visitadas. Uma estratégia de exploração de verdade precisaria de memória, por exemplo uma grade das regiões já visitadas, o que exige localização.

A segunda limitação é o sensor. Os feixes estão separados por 45°, e um obstáculo estreito entre dois feixes pode passar despercebido até ficar muito perto. Na arena do exercício isso não ocorre, porque as paredes e o cilindro de 0,30 m de diâmetro são grandes, mas um obstáculo fino ou outro robô poderiam escapar. O alcance de 0,5 m também limita a velocidade segura, já que o robô precisa frear e girar dentro desse alcance.

A terceira é a dependência dos parâmetros em relação ao ambiente. Em um corredor de largura w, os feixes diagonais medem $w/\sqrt{2}$ quando o robô está no centro. Em passagens mais estreitas que $\sqrt{2}\,d_{\text{giro}} \approx 0{,}18\,\mathrm{m}$ na configuração A, o modo de escape seria acionado e o robô não passaria. Além disso, o escape gira sempre para o lado escolhido na entrada, o que resolve os cantos da arena, mas não garante a saída de becos mais profundos que o alcance do lidar.

5.2 Limitações da navegação até metas

O controlador do `goto` não usa os sensores de distância, então colidiria com qualquer obstáculo entre o robô e a meta. Na arena vazia do exercício isso não acontece, mas basta um obstáculo para exigir a combinação com o `avoid`. Ele também não controla a orientação final do robô, o que não importa aqui, mas seria necessário em tarefas como empurrar um objeto em uma direção.

A limitação mais séria é a dependência da odometria. No simulador, a pose publicada é a verdadeira, e o robô atingiu as metas sem nenhum erro de localização. No robô real, a meta seria atingida na posição estimada, e o erro da odometria se somaria ao erro de chegada.

Por fim, o desempenho está limitado pela velocidade linear máxima, e não pelos ganhos. O ajuste $\Delta t \approx 4{,}99\,\rho_0 + 0{,}29\,|\alpha_0| - 0{,}20$ e uma conta simples mostram isso. No conjunto A, o tempo mínimo para percorrer em linha reta, a $v_{\max}$, a distância até cada meta corresponde a 86 % do tempo total dos ensaios, e sobra pouco a ganhar com o giro. Não houve órbita em torno das metas, porque perto delas v cai proporcionalmente a ρ enquanto ω continua proporcional a α, e o robô corrige a direção antes de passar pela meta. Com ganhos k_ρ maiores ou sem a redução por ρ, porém, o robô chegaria rápido demais e

poderia contornar a meta sem entrar no raio de 6 cm.

5.3 Falhas observadas

Tabela 7. Casos em que o robô falhou durante o desenvolvimento e os ensaios.

Nó	O que aconteceu	Causa provável e correção	Evidência
`avoid`	órbita fixa em torno do cilindro, na simulação cinemática preliminar	a lei centralizava o robô no corredor entre cilindro e paredes. Resolvido com d_{inf}	simulação offline
`avoid`	circuito fechado quase quadrado a partir de 30 s, com cobertura de 31,7 %	dinâmica reativa determinística com um ciclo atrator. Resolvido com o viés de vagar	Figura 3
`avoid`	na configuração B, $d_{\min}$ chegou a 0,065 m, cerca de 2 cm do limiar, em um canto	frenagem e escape tardios. Não colidiu, mas a margem é pequena para o robô real	Figura 6
`goto`	no conjunto B, arcos largos quando a meta aparece atrás do robô, com até 15 % de caminho a mais	alinhamento lento com $k_\alpha = 1$. Corrigido no conjunto A	Figura 9

5.4 Transferência para o HeRo real

Os tópicos e os tipos de mensagem do simulador são os mesmos do robô real, mas o kit faz algumas simplificações que não valem para o HeRo físico. No simulador, a odometria publicada é a pose verdadeira do robô e o lidar é um traçado de raios sem ruído, como se vê em `sim/psr_sim/world.py`. Além disso, o tópico `/hero_0/goal` é publicado pelo próprio juiz do cenário.

Para rodar o mesmo código no HeRo físico, seriam necessárias mudanças em cinco frentes.

Sensores. Os sensores reais têm ruído, leituras espúrias e um alcance efetivo que pode ser menor que 0,5 m, dependendo da superfície do obstáculo. Seria preciso filtrar as leituras, por exemplo com a mediana das últimas amostras de cada feixe, e rever os limiares. A configuração A deixou no pior caso cerca de 6,5 cm entre o corpo e o obstáculo, o que acomoda algum ruído. A configuração B, com cerca de 3 cm, teria pouca margem.

Odometria. No `goto`, a pose viria da odometria das rodas, que acumula erro com o escorregamento. Como cada meta é perseguida a partir da pose estimada, o erro cresceria de meta em meta, e depois de algumas dezenas de metas o robô chegaria a pontos deslocados. Seria preciso um sistema de localização externo para corrigir a pose ou, ao menos, para medir o erro.

Atuadores. Os motores reais têm zona morta e limite de aceleração. A lei do `goto` comanda velocidades pequenas perto da meta (a 6 cm, $k_\rho\,\rho$ vale 0,09 m/s) e durante o alinhamento, quando $\cos\alpha$ é pequeno, e essas velocidades podem não mover o robô. Pode ser preciso impor uma velocidade mínima ou compensar a zona morta.

Comunicação. O comando passa por uma rede sem fio. Atrasos variáveis reduzem a margem efetiva do `avoid`, e perdas de pacotes por mais de 0,5 s param o robô pelo watchdog. O laço a 20 Hz tem folga para isso, mas a latência real precisaria ser medida.

Metas e tópicos. O tópico `/hero_0/goal` é publicado pelo juiz do simulador e não existe no robô real, então as metas teriam de vir de outro nó. O `hero_bridge` também assina `velocity_controller/cmd_vel` no namespace do robô, o que sugere conferir qual tópico o firmware do HeRo real usa para o comando. Com mais de um robô, basta ajustar o parâmetro `~robot`, que o código já usa para montar os nomes dos tópicos.

6. Conclusão

Este trabalho implementou e avaliou dois comportamentos para o HeRo em ROS, usando o simulador do kit. No desvio de obstáculos, uma lei reativa com saturação do alcance de influência, escape com histerese e viés de vagar manteve o robô em movimento por 5 min sem nenhuma colisão, nas duas configurações e nos seis ensaios. O viés se mostrou essencial: sem ele, o robô ficou preso em um circuito fechado e cobriu só 31,7 % da arena, contra 56,8 % com ele. A configuração B percorreu 35 % a mais e cobriu 76,2 % da arena, ao custo de reduzir a menor distância aos obstáculos de 0,106 m para 0,067 m.

Na navegação até metas, o controle proporcional em α com a velocidade linear reduzida por $\cos\alpha$ atingiu 33,3 metas em 3 min com o conjunto A, a 5,32 s por meta, e o tempo por meta ficou limitado essencialmente pela velocidade linear máxima do robô. Reduzir k_α de 3 para 1 custou cerca de 15 % das metas, por causa do alinhamento mais lento e dos arcos mais largos.

Para o robô real, a equipe usaria a configuração A no `avoid`, pela margem de segurança, e o conjunto A no `goto`. O principal aprendizado é que comportamentos reativos simples funcionam bem no simulador, mas podem ter ciclos que só aparecem quando a trajetória é medida e plotada, e que as maiores dificuldades da transferência estão nas simplificações do simulador, como a odometria perfeita e o lidar sem ruído. O próximo passo é testar os dois nós no HeRo físico e combiná-los para navegar até metas desviando de obstáculos.

A. Reprodução dos ensaios

O repositório `https://github.com/TulioHRC/robotica-cain` é o kit da disciplina acrescido dos arquivos da equipe. O README do kit foi mantido como `README-kit.md`, e o `README.md` da raiz descreve a reprodução dos ensaios. Os arquivos da equipe são os seguintes:

```
exercicios/
  avoid/   avoid.py   avoid_b.py   avoid_sem_vies.py
           logs/avoid_A_*.csv   logs/avoid_B_*.csv   logs_sem_vies/
  goto/    goto.py    goto_b.py    logs/goto_A_*.csv    logs/goto_B_*.csv
analise/   comum.py   plot_avoid.py   plot_goto.py   requirements.txt
relatorio/ relatorio5.tex   figs/   dados/
```

As configurações B são subclasses das classes de A que só redefinem os parâmetros. O mesmo vale para `avoid_sem_vies.py`, que zera o viés de vagar e grava `avoid_SV_*.csv`. O ensaio sem viés da Figura 3 foi gravado com o nome da configuração A e guardado em `logs_sem_vies/`, para não entrar nas médias de A. Para repetir os ensaios do `avoid`:

```
./run.sh down
./run.sh run avoid
./run.sh down
./run.sh run avoid script:=avoid_b.py
```

Os ensaios do `goto` são iguais, com `goto` e `script:=goto_b.py`. Para gerar as figuras e as tabelas, fora do container:

```
pip install -r analise/requirements.txt
python3 analise/plot_avoid.py
python3 analise/plot_goto.py
```

Por padrão, os scripts usam todos os ensaios completos de cada configuração encontrados nas pastas `logs/`, gravam as figuras em `relatorio/figs/` e as tabelas de métricas em `relatorio/dados/`, que este

documento inclui automaticamente.

Referências

[1] P. Rezeck, H. Azpúrua, M. F. S. Corrêa e L. Chaimowicz. HeRo 2.0: a low-cost robot for swarm robotics research. *Autonomous Robots*, 47:879–903, 2023. https://doi.org/10.1007/s10514-023-10100-0

[2] VeRLab/UFMG. *hero_common*: pacotes ROS e modelos do HeRo. https://github.com/verlab/hero_common

[3] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler e A. Y. Ng. ROS: an open-source Robot Operating System. In *ICRA Workshop on Open Source Software*, 2009.

[4] Genesis Authors. *Genesis: a generative and universal physics engine for robotics and beyond*, 2024. https://genesis-world.readthedocs.io/

[5] R. Siegwart, I. R. Nourbakhsh e D. Scaramuzza. *Introduction to Autonomous Mobile Robots*. 2. ed. MIT Press, 2011.